

Java 8 Streams — Theory Guide

Core concepts you need to answer the interview question bank.

1. What Is a Stream?

A Stream is a sequence of elements from a source (collection, array, generator, I/O channel) that supports functional-style operations — filter, map, reduce, etc. — computed as a pipeline.

Key differences from a Collection:

- **Not a data structure.** A Stream doesn't store elements; it computes them on demand from a source and passes them through a pipeline.
- **Functional in nature.** Operations produce a result but don't modify the source.
- **Lazy.** Intermediate operations don't run until a terminal operation is invoked.
- **Possibly unbounded.** Stream.iterate() / Stream.generate() can produce infinite streams; limit() bounds them.
- **Consumable once.** A Stream can only be traversed once. After a terminal operation runs, that stream instance is "closed" — reusing it throws IllegalStateException: stream has already been operated upon or closed.
- **Not thread-safe as a shared mutable object**, but a *parallel* stream can safely split work across threads internally, provided your lambdas are stateless and non-interfering.

2. The Pipeline: Source → Intermediate Ops → Terminal Op

```
source.stream()  
  .filter(...)    // intermediate — returns a Stream  
  .map(...)       // intermediate — returns a Stream  
  .collect(...)   // terminal — triggers execution, returns a result
```

- **Intermediate operations** (filter, map, flatMap, distinct, sorted, peek, limit, skip) are lazy and return a new Stream. They are only *recorded*, not executed.
- **Terminal operations** (forEach, collect, reduce, count, anyMatch/allMatch/noneMatch, findFirst/findAny, toArray, min/max) trigger a single pass through the pipeline and produce a result or side effect. After a terminal op runs, the stream is consumed.

Why laziness matters: because operations aren't run until a terminal op is called, the JVM can fuse operations into a single pass per element (rather than materializing an intermediate list at every step), and it can **short-circuit** — stop early — for operations like findFirst(), anyMatch(), or limit(), even on infinite streams.

3. Creating Streams

Method

Example

From a Collection	<code>list.stream()</code>
From varargs	<code>Stream.of(1, 2, 3)</code>
From an array	<code>Arrays.stream(arr)</code>
Infinite, function-based	<code>Stream.generate(Math::random)</code>
Infinite, seed + function	<code>Stream.iterate(1, n -> n * 2)</code>
Numeric range	<code>IntStream.range(0, 5)</code> (exclusive), <code>IntStream.rangeClosed(0, 5)</code> (inclusive)
From a file	<code>Files.lines(path)</code>
From a regex split	<code>pattern.splitAsStream(str)</code>
Empty	<code>Stream.empty()</code>
Concatenation	<code>Stream.concat(s1, s2)</code>

`Stream.generate(Supplier)` produces elements with no relation to prior ones (e.g., random numbers); `Stream.iterate(seed, UnaryOperator)` produces each element from the previous one (e.g., powers of two). Both are infinite unless bounded with `limit()`.

Primitive streams (`IntStream`, `LongStream`, `DoubleStream`) exist to avoid the cost of autoboxing `Integer`/`Long`/`Double` on every element, and they expose numeric-only terminal ops like `sum()`, `average()`, and `summaryStatistics()`.

4. Intermediate Operations in Detail

- **`filter(Predicate<T>)`** — keeps elements matching the predicate.
- **`map(Function<T,R>)`** — transforms each element 1-to-1.
- **`flatMap(Function<T, Stream<R>>)`** — transforms each element into a `Stream`, then flattens all those streams into one. Used to turn a `Stream<List<T>>` (or any nested structure) into a flat `Stream<T>`.
- **`distinct()`** — removes duplicates using `equals()`/`hashCode()`.
- **`sorted()` / `sorted(Comparator)`** — natural ordering (requires `Comparable`) or a custom comparator.
- **`peek(Consumer)`** — runs a side-effect action per element without changing the stream; intended for debugging, not business logic, because with laziness/short-circuiting it may not run for every element, and its execution order is unspecified with parallel streams.
- **`limit(n)`** — truncates to the first `n` elements (short-circuiting).
- **`skip(n)`** — discards the first `n` elements. `skip(n).limit(m)` implements simple pagination.

5. Terminal Operations in Detail

- **`forEach(Consumer)` / `forEachOrdered(Consumer)`** — performs an action per element. `forEach` doesn't guarantee encounter order in parallel streams; `forEachOrdered` does (at the cost of parallel speed-up).
- **`collect(Collector)`** — accumulates elements into a result container (list, map, string, custom object). See Collectors below.

- **reduce(...)** — combines elements into a single value. Three overloads:
 1. `reduce(BinaryOperator<T>)` → returns `Optional<T>` (empty stream → empty `Optional`).
 2. `reduce(T identity, BinaryOperator<T>)` → returns `T` (identity is the seed and the fallback for an empty stream).
 3. `reduce(U identity, BiFunction<U,T,U> accumulator, BinaryOperator<U> combiner)` → used when the result type differs from the element type; the combiner merges partial results and only matters in parallel execution.
- **count()** — number of elements.
- **anyMatch()** / **allMatch()** / **noneMatch()** — short-circuiting boolean checks. On an empty stream: `anyMatch` → false, `allMatch` → true, `noneMatch` → true (vacuous truth).
- **findFirst()** / **findAny()** — return `Optional<T>`. `findFirst()` always returns the first element in encounter order (even in parallel); `findAny()` is unordered and can return as soon as any thread finds a match, so it's faster in parallel streams.
- **min(Comparator)** / **max(Comparator)** — return `Optional<T>`.
- **toArray()** — collects into an array.

Short-circuiting operations: `limit`, `findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch` — these can terminate processing before visiting every element (essential for working with infinite streams).

6. Collectors

Collectors (a factory class) supplies ready-made Collector implementations for `collect()`:

- **toList()** / **toSet()** / **toUnmodifiableList()** — basic accumulation.
- **toMap(keyFn, valueFn)** — throws `IllegalStateException` on duplicate keys unless you supply a third argument, a merge `BinaryOperator<V>` (`toMap(keyFn, valueFn, (v1, v2) -> v1)`), and optionally a fourth `Supplier<Map>` for the map implementation.
- **groupingBy(classifier)** — returns `Map<K, List<T>>`. `groupingBy(classifier, downstream)` applies a further collector to each group (e.g., `groupingBy(Employee::getDept, counting())` or `groupingBy(Employee::getDept, mapping(Employee::getName, toList()))`). Nested `groupingBy` calls produce a multi-level map.
- **partitioningBy(predicate)** — like `groupingBy` but always splits into exactly two groups keyed by `Boolean` (true/false).
- **joining()** / **joining(delimiter)** / **joining(delimiter, prefix, suffix)** — concatenates `CharSequence` elements.
- **counting()**, **summingInt/Long/Double()**, **averagingInt/Long/Double()**, **summarizingInt/Long/Double()** — numeric reductions, often used as a `groupingBy` downstream.

- **mapping(function, downstream)** — transforms elements before applying another collector; commonly paired with groupingBy.
- **reducing(...)** — a collector-based version of reduce(), useful as a downstream collector.
- **collectingAndThen(collector, finisher)** — wraps a collector's result with an extra transform, e.g. wrapping a mutable list into an immutable one.

How a Collector works internally — four components:

1. **Supplier** — creates a new mutable result container (e.g., ArrayList::new).
2. **Accumulator** — folds an element into that container (e.g., List::add).
3. **Combiner** — merges two containers (needed for parallel execution).
4. **Finisher** (*optional*) — transforms the intermediate container into the final result.

Characteristics (Collector.Characteristics): CONCURRENT (the container can be shared/updated concurrently), UNORDERED (encounter order doesn't matter), IDENTITY_FINISH (the finisher is the identity function, so the container itself is the result — allows the JDK to skip calling the finisher).

You can build a custom collector with Collector.of(supplier, accumulator, combiner, [finisher], characteristics...).

7. Optional

Optional<T> is a container that may or may not hold a non-null value; it exists to make the possible absence of a value explicit in the type system and reduce NullPointerExceptions.

- **Optional.of(value)** — throws NullPointerException immediately if value is null.
- **Optional.ofNullable(value)** — wraps a possibly-null value safely.
- **Optional.empty()** — an explicitly empty Optional.
- **orElse(default)** — returns the value, or default if empty. **default is always evaluated eagerly**, even when the Optional is present — wasteful if it's an expensive computation.
- **orElseGet(Supplier)** — like orElse but the supplier only runs if the Optional is empty (lazy) — preferred for expensive defaults.
- **orElseThrow(...)** — throws a supplied exception (or NoSuchElementException with no args) if empty.
- **map(Function) / flatMap(Function)** — transform the contained value if present, otherwise stay empty; flatMap is used when the mapping function itself returns an Optional, to avoid Optional<Optional<T>>.
- **ifPresent(Consumer) / ifPresentOrElse(Consumer, Runnable)** — run an action conditionally, avoiding the isPresent() + get() anti-pattern.
- **stream()** (*Java 9+*) — converts Optional<T> to a Stream<T> of 0 or 1 elements, handy for flat-mapping a stream of Optionals.

`findFirst()`, `findAny()`, `min()`, `max()`, and single-value `reduce()` all return `Optional` because the stream might be empty.

Best practice: don't use `Optional` as a method parameter, constructor parameter, or class field — it's designed as a *return type* to signal “this may be absent” to callers; using it elsewhere adds serialization/boilerplate overhead without benefit.

8. Parallel Streams

`collection.parallelStream()` or `stream.parallel()` splits the pipeline's work across multiple threads using the **Fork/Join framework**, specifically `ForkJoinPool.commonPool()`, whose default thread count is `Runtime.getRuntime().availableProcessors() - 1`.

Parallel streams help when:

- The dataset is large.
- The per-element work is CPU-bound and substantial.
- The operations are stateless, non-interfering, and associative (required for correct `reduce/collect` combiner behavior).

Parallel streams tend to hurt when:

- The dataset is small (thread coordination overhead dominates).
- The source doesn't split efficiently (e.g., `LinkedList` or `Stream.iterate()` split poorly compared to `ArrayList` or arrays).
- Operations involve I/O or blocking calls (threads sit idle, starving the shared common pool for the whole JVM).
- Shared mutable state is touched from `peek()/forEach()` — this introduces race conditions since there's no synchronization guarantee.
- Order matters and you use `forEachOrdered` or `findFirst` (these give up most of the parallel speed-up by re-imposing ordering).

`findAny()` is faster than `findFirst()` in parallel because it can return as soon as *any* thread finds a match; `findFirst()` must respect encounter order, so it may need to wait for earlier segments to finish even if a later segment matches sooner.

The accumulator and combiner functions passed to `reduce/collect` must be **associative** (grouping order doesn't change the result) and ideally **stateless**, because the JDK may split the stream into chunks, reduce each chunk independently on different threads, and then combine the partial results in any order.

You can run a stream in an isolated thread pool (rather than the shared common pool) by submitting it inside `new ForkJoinPool(n).submit(() -> stream.parallel()).get()`.

9. Functional Interfaces & Method References

Core `java.util.function` interfaces used throughout the Stream API:

Interface	Signature	Used by
Predicate<T>	boolean test(T t)	filter, anyMatch, etc.
Function<T,R>	R apply(T t)	map
BiFunction<T,U,R>	R apply(T t, U u)	reduce (3-arg form), Map.merge
Consumer<T>	void accept(T t)	forEach, peek
Supplier<T>	T get()	Stream.generate, orElseGet
UnaryOperator<T>	T apply(T t) — Function<T,T>	Stream.iterate
BinaryOperator<T>	T apply(T t1, T t2) — BiFunction<T,T,T>	reduce (1- and 2-arg forms)

Method references — four forms, all shorthand for a lambda:

1. Static: `Integer::parseInt` $\equiv s \rightarrow \text{Integer.parseInt}(s)$
2. Instance method of a particular object: `myList::add` $\equiv x \rightarrow \text{myList.add}(x)$
3. Instance method of an arbitrary object of a type (used implicitly as the first lambda parameter): `String::toUpperCase` $\equiv s \rightarrow \text{s.toUpperCase}()$
4. Constructor: `ArrayList::new` $\equiv () \rightarrow \text{new ArrayList}(<>())$

Composing Predicates/Functions:

- `Predicate.and()`, `.or()`, `.negate()` combine filter conditions, e.g. `isAdult.and(isActive)`.
- `Function.andThen()` runs the current function then the argument; `Function.compose()` runs the argument first, then the current function.
- `Comparator.comparing(keyExtractor)`, `.thenComparing(...)`, `.reversed()` build multi-field comparators for `sorted()`, e.g. `Comparator.comparing(Employee::getDept).thenComparing(Employee::getSalary, Comparator.reverseOrder())`.

10. Common Pitfalls & Gotchas

- **Lambdas can only capture effectively-final local variables** — a variable that's never reassigned after initialization. This is required because the lambda may run on a different thread/at a different time than the enclosing method's stack frame.
- **ConcurrentModificationException** is thrown if you try to structurally modify a collection while a stream sourced from it is being traversed.
- **Checked exceptions** can't be thrown directly from most functional interface methods (they don't declare throws); you must catch-and-wrap, use a helper that rethrows as unchecked, or use a custom functional interface that declares the checked exception.
- **Collectors.toList() mutability**: in Java 8, the list returned by `Collectors.toList()` happens to be mutable (`ArrayList`) but this isn't guaranteed by the API contract. Java 16+ added `Stream.toList()`, which explicitly returns an **immutable** list.

- **Autoboxing cost:** using `Stream<Integer>` for numeric work boxes/unboxes every element; `IntStream/LongStream/DoubleStream` avoid this and should be preferred for numeric-heavy pipelines (`mapToInt`, `mapToObj` convert between the two).
- **Internally, a Stream pipeline is implemented as a chain of Sink objects** — each intermediate operation wraps the next sink, forming a chain that processes one element at a time when the terminal operation finally pulls/pushes elements through it. This is why the JDK can fuse `filter+map+etc.` into a single pass instead of materializing a list at each stage — conceptually similar to the Decorator pattern.